

PROJET ANALYSE NUMERIQUE

Resolution de systeme lineaire par la methode de Gauss-Seidel

Encadrant :
M. J.G.CAPUTO

Présenté par :
Mouad SHERADJ
DRISSI
Amr ZAKI SALIH

Remerciements

Nous tenons à remercier Monsieur Caputo pour son accompagnement et pour l'opportunité qu'il nous a offerte à travers ce projet d'analyse numérique. Ce travail nous a permis de mettre en pratique les concepts théoriques étudiés en classe, tout en approfondissant notre compréhension de l'importance des méthodes numériques dans la résolution de problèmes complexes. Cette expérience a également renforcé nos compétences en programmation et en analyse mathématique, contribuant ainsi à notre développement académique et professionnel.

Table des matières

1	Partie 1	3
1.1	Méthodes itératives	3
1.2	Méthode de Gauss-Seidel	4
1.3	Algorithme de résolution	5
1.4	Critère d'arrêt	6
2	Partie 2	8
2.1	Test et Interprétation	8

Introduction

Ce rapport présente une étude approfondie des méthodes itératives de résolution de systèmes linéaires. Plus particulièrement, il se concentre sur l'utilisation de la méthode de Gauss-Seidel. Ces méthodes jouent un rôle crucial dans les applications scientifiques où les systèmes d'équations sont souvent trop complexes pour être résolus directement, notamment en raison de la taille des matrices impliquées.

L'objectif de ce travail est d'analyser la méthode de Gauss-Seidel, en comparant l'efficacité de cette dernière dans des configurations variées. Pour cela, nous avons exploré des cas pratiques, en testant différentes tailles de matrices, tolérances, et ajustements des paramètres.

Nous commencerons par une définition des méthodes itératives et de la méthode de Gauss-Seidel, puis définirons les algorithmes et discuterons ensuite des implémentations pratiques en Python, Matlab et Fortran. Les résultats obtenus à partir de ces implémentations seront ensuite analysés.

1 Partie 1

1.1 Méthodes itératives

Une méthode de résolution du système $Ax = b$, où A appartient à $M_n(\mathbb{R})$ est inversible et b appartient à $\mathbb{R}^n$, est dite itérative si elle converge vers la solution x après un nombre infini d'itérations. À chaque étape, on calcule le résidu du système. Dans le cas d'une matrice pleine, le coût de chaque itération est de l'ordre de n^2 , tandis que pour une méthode directe, le coût global est d'environ $\frac{2}{3}n^3$. Ainsi, les méthodes itératives peuvent s'avérer avantageuses par rapport aux méthodes directes.

La résolution du système par une méthode itérative repose sur la construction d'une suite convergente de vecteurs x^k telle que :
 $x = \lim_{k \rightarrow \infty} x^k$ où x est la solution recherchée. Autrement dit, les méthodes itératives visent à obtenir une solution approchée.

On pose $A = M - N$, avec M : une matrice facile à inverser.

$$Ax = b \Leftrightarrow (M - N)x = b \Leftrightarrow Mx = Nx + b \Leftrightarrow x = M^{-1}(Nx + b)(1)$$

En définissant $\Phi(x) = M^{-1}(Nx + b)$.

Si Φ est une contraction stricte, alors la suite définie par :

$$\begin{cases} x_0 \in \mathbb{R}^n \\ x_{n+1} = \Phi(x_n) \end{cases}$$

converge vers x^* , solution de $Ax = b$.

La forme itérative de ce schéma est donc la suivante :

$$\begin{cases} x_0 \in \mathbb{R}^n \\ x_{n+1} = M^{-1}(Nx_n + b) \end{cases}$$

où la suite (x_n) converge vers x^* , la solution du système $Ax = b$. (d'après theoreme du point fixe...)

$$A = D + L + U,$$

où :

- D est la matrice diagonale de A ,
- L est la matrice triangulaire inférieure stricte (les éléments en dessous de la diagonale),
- U est la matrice triangulaire supérieure stricte (les éléments au-dessus de la diagonale).

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \in M_n(\mathbb{R})$$

$$= L + D + U$$

$$= \begin{pmatrix} 0 & 0 & \cdots & 0 \\ a_{21} & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ a_{n1} & \cdots & a_{n,n-1} & 0 \end{pmatrix} + \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & a_{n-1,n} \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

1.2 Méthode de Gauss-Seidel

La méthode de Gauss-Seidel est une méthode itérative qui repose sur la décomposition de la matrice A et les définitions suivantes :

$$M = D + L \quad \text{et} \quad N = -U.$$

En remplaçant dans l'expression (1) :

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b,$$

On arrive à l'équation matricielle suivante :

$$x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b.$$

Condition de convergence

Pour que la méthode de Gauss-Seidel converge, il est nécessaire que la matrice A soit à diagonale strictement dominante. Cela signifie que pour chaque ligne i :

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|,$$

où $|a_{ii}|$ est la valeur absolue de l'élément diagonal, et $\sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ est la somme des valeurs absolues des éléments non diagonaux de la ligne i .

1.3 Algorithme de résolution

Considérons le système linéaire $Ax = b$, où A est une matrice de coefficients, x est le vecteur des inconnues, et b est le vecteur des constantes. Le système peut être écrit sous forme développée comme suit :

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n. \end{cases}$$

Les inconnues $x_1, x_2, \dots, x_n$ peuvent être exprimées de manière explicite en isolant chaque variable en fonction des autres. Par exemple :

$$x_1 = \frac{b_1 - (a_{12}x_2 + \cdots + a_{1n}x_n)}{a_{11}},$$

$$x_2 = \frac{b_2 - (a_{21}x_1 + \cdots + a_{2n}x_n)}{a_{22}},$$

$\vdots$

$$x_n = \frac{b_n - (a_{n1}x_1 + \cdots + a_{n(n-1)}x_{n-1})}{a_{nn}}.$$

La formule générale pour calculer x_i s'écrit :

$$x_i = \frac{b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j}{a_{ii}}.$$

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} a_{i,j}x_j - \sum_{j=i+1}^n a_{i,j}x_j}{a_{i,i}}.$$

Version itérative de la méthode de Gauss-Seidel

Dans la méthode de Gauss-Seidel, la formule devient :

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{i,j}x_j^{(k+1)} - \sum_{j=i+1}^n a_{i,j}x_j^{(k)}}{a_{i,i}}.$$

La méthode itérative complète peut être exprimée sous forme d'algorithme. Voici une implémentation en pseudo-code :

Algorithme : Gauss-Seidel

```

1: Procédure Gauss-Seidel(E :  $A$  : tableau  $[1..max, 1..max]$  d'entiers ;  $b$  : tableau  $[1..max]$  d'entiers ;  $k, n$  : entiers ; E/S :  $x$  : tableau  $[1..max]$  réels, erreur : réel)
2: Var  $i, j, m, a, l$  : entier
3:    $y$  : tableau  $[1..max]$  réels
4: Début
5:   erreur  $\leftarrow \epsilon + 1$ 
6:   Pour  $i \leftarrow 1$  jusqu'à  $k$  Faire
7:     Si erreur  $> \epsilon$  Alors
8:       Pour  $j \leftarrow 1$  jusqu'à  $n$  Faire
9:          $y[j] \leftarrow b[j]$ 
10:        Pour  $m \leftarrow 1$  jusqu'à  $j - 1$  Faire
11:           $y[j] \leftarrow y[j] + A[j, m] \times y[m]$ 
12:        FinPour
13:        Pour  $a \leftarrow j + 1$  jusqu'à  $n$  Faire
14:           $y[j] \leftarrow y[j] + A[j, a] \times y[a]$ 
15:        FinPour
16:         $y[j] \leftarrow y[j] / A[j, j]$ 
17:      FinPour
18:      erreur  $\leftarrow \max(\text{abs}(y - x))$ 
19:      Pour  $l \leftarrow 1$  jusqu'à  $n$  Faire
20:         $x[l] \leftarrow y[l]$ 
21:      FinPour
22:    Finsi
23:  FinPour
24: Fin

```

1.4 Critère d'arrêt

Le critère d'arrêt est la condition qui permet de savoir si une approximation $x^{(k)}$ est suffisamment proche de la solution exacte du système linéaire selon une tolérance définie

à l'avance.

1.4.1 Critère basé sur l'incrément

Ce critère repose sur la variation entre deux itérations consécutives $x^{(k)}$ et $x^{(k+1)}$. Le programme s'arrête lorsque la norme de cette différence devient suffisamment petite, c'est-à-dire :

$$\|x^{(k+1)} - x^{(k)}\| < \text{tol} \quad \text{avec} \quad \text{tol} > 0.$$

Analyse du critère

Ce critère possède une limite. Considérons la suite de solutions x_k suivante et r , la solution exacte du système $Ax = b$:

$$\begin{cases} x^{(0)} \in \mathbb{R}^n, \\ x^{(k+1)} = g(x^{(k)}), \end{cases}$$

où g est une fonction d'itération.

On a alors :

$$\|x^{(k+1)} - x^{(k)}\| = \|x^{(k+1)} - r + r - x^{(k)}\| = \|g(x^{(k)}) - r + r - x^{(k)}\|.$$

Cela peut être réécrit comme :

$$\|x^{(k+1)} - x^{(k)}\| = \|x^{(k)} - r\| \cdot \|g(x^{(k)}) - 1\|.$$

En appliquant le théorème des accroissements finis, on obtient :

$$\|g(x^{(k)}) - 1\| = \left\| \frac{g(x^{(k)}) - 1}{x^{(k)} - r} - 1 \right\| \cdot \|x^{(k)} - r\| = \|g'(\zeta) - 1\| \cdot \|x^{(k)} - r\|,$$

avec ζ appartenant à un voisinage de r .

- Si $g'(r) \simeq 0$, alors le test est pertinent, car la distance entre deux itérations successives est presque égale à la distance entre l'itération et la racine.

- Si $\|g'(r) - 1\| \ll 1$, c'est-à-dire $g'(r) \simeq 1$, alors $\|x_{n+1} - x_n\|$ peut atteindre la précision voulue sans garantir que l'erreur réelle $e^{(k)} = \|x^{(k)} - r\|$ soit suffisamment petite ce qui fausse le résultat final.

1.4.2 Critère basé sur le résidu

Un résidu $r^{(k)}$ mesure l'erreur dans l'équation $Ax = b$, il est défini comme suit :

$$r^{(k)} = b - Ax^{(k)}.$$

On arrête les itérations lorsque la norme du résidu devient suffisamment petite, c'est-à-dire :

$$\frac{\|r^{(k)}\|}{\|b\|} < \text{tol}, \quad \text{avec} \text{tol} > 0,$$

Analyse du critère

On a :

$$\frac{\|x - x^{(k)}\|}{\|x\|} \leq \text{cond}(A) \cdot \frac{\|r^{(k)}\|}{\|b\|}.$$

Si la matrice A est bien conditionnée, alors le résidu est une bonne mesure de l'erreur relative entre $x^{(k)}$ et la solution exacte r .

Sinon, si la matrice est mal conditionnée, alors le résidu va ne pas être un indicateur fiable de l'erreur.

2 Partie 2

2.1 Test et Interprétation

Avant de s'attaquer réellement au code Fortran, il était préférable de prototyper le code dans des langages ayant une syntaxe plus simple et intuitive tel que python et matlab. En effet, le Fortran possède une syntaxe stricte ainsi que minimaliste et est orientée performance. Cependant ce dernier est très rapide pour les calculs numériques compilés, alors que le python par exemple, lui, est beaucoup plus lent sans optimisation. Le Fortran permet donc des performances optimales pour des simulations scientifiques.

Ainsi pour chacun des langages de programmation, nous avons conçu deux programmes qui utilisent deux méthodes différentes pour réaliser Gauss Seidel. L'une d'elle se code comme une itération du type :

$$X_{k+1} = FX_k + G$$

similairement à la méthode de Jacobi et où $F = -(D + L)^{-1}U$ et $G = (D + L)^{-1}B$ et que l'on appellera **programme A**.

L'autre quant à elle utilise directement

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{i,j}x_j^{(k+1)} - \sum_{j=i+1}^n a_{i,j}x_j^{(k)}}{a_{i,i}}.$$

sans se soucier de problème d'optimisation que l'on appellera **programme B**

Pour chacune des résolutions de Gauss Seidel, nous allons ainsi utiliser les paramètres suivants :

- $n = 3$
- $n = 5$
- $n = 10$

Pour la taille de la matrice de Hilbert.

- $\omega = 0.5$
- $\omega = 1$
- $\omega = 1.5$

Pour l'ajustement de la diagonale de A et pour ainsi faciliter la convergence.

- $\epsilon = 1e-4$
- $\epsilon = 1e-6$
- $\epsilon = 1e-8$

Pour la tolérance.

On fixe par ailleurs le nombre maximal d'itérations à $itermax = 20000$ sans quoi il n'y a pas convergence vers une solution.

2.1.1 Python

Nous avons testé les performances des deux programmes A et B avec des matrices de Hilbert de tailles différentes ($n = 2, 3, \dots, 10$), avec $\omega = 0.5$ et $\epsilon = 1e-6$. Les résultats sont présentés dans les deux graphiques ci-dessous : le premier représente le temps d'exécution des deux méthodes, tandis que le second illustre le nombre d'itérations nécessaires pour atteindre la convergence.

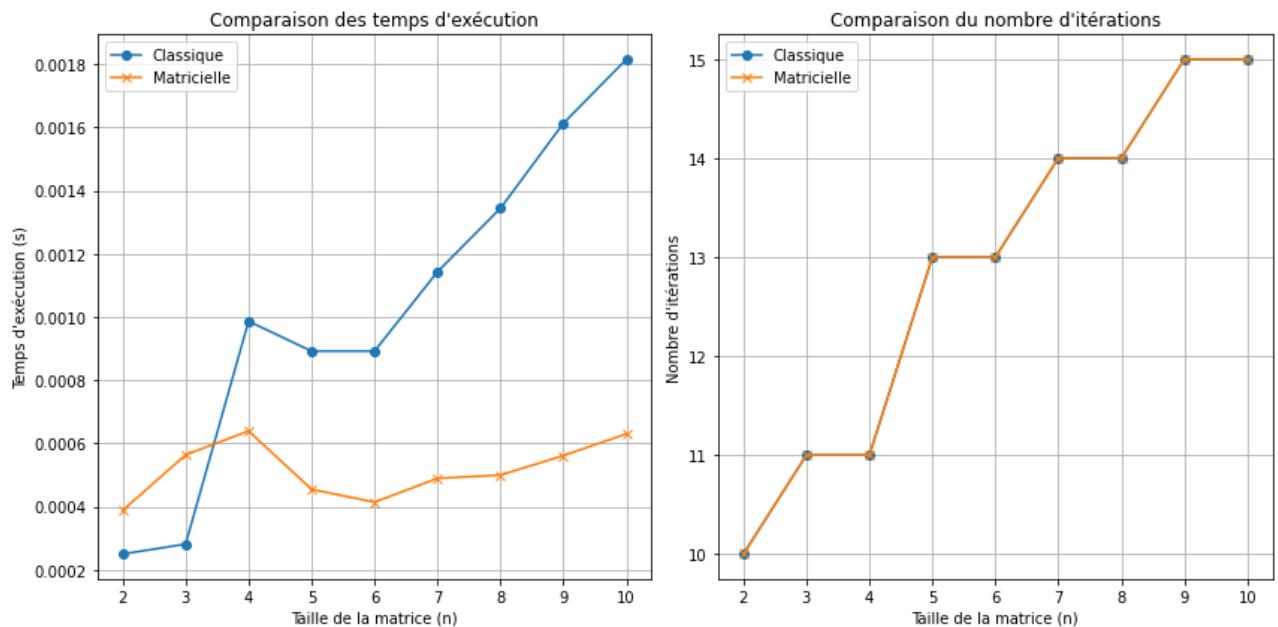


FIGURE 1.1 – Comparaison des temps d'exécution et du nombre d'itérations entre les programmes A et B en Python.

Les résultats montrent que le programme A est significativement plus rapide que le programme B, grâce à l'utilisation des fonctionnalités optimisées de Python et de ses

bibliothèques comme NumPy. Le programme B repose sur des boucles élément par élément, ce qui devient de plus en plus coûteux en termes de temps d'exécution à mesure que la taille de la matrice augmente. En revanche, le **nombre d'itérations nécessaires à la convergence** est identique pour les deux programmes, car ils appliquent le même algorithme de Gauss-Seidel pour la mise à jour des approximations. L'approche matricielle s'avère nettement plus performante et adaptée aux matrices de grande taille.

2.1.2 Matlab

Matlab est un langage interprété qui permet le calcul scientifique, l'analyse de données ainsi que la modélisation et le développement d'algorithmes. Dans cette partie, nous allons tester pour chacun des deux programmes matlab différents paramètres de tolérance, de ω ainsi que de taille de matrice n .

On commence ainsi par le **programme A**

Pour $n = 3, \omega = 0.5, \epsilon = 1e - 4$, on obtient que la convergence est atteinte en 8 itérations.

Pour $n = 5, \omega = 1, \epsilon = 1e - 6$, on obtient que la convergence est atteinte en 10 itérations.

pour $n = 10, \omega = 1.5, \epsilon = 1e - 8$, on obtient que la convergence est atteinte en 12 itérations.

Ensuite on a le **programme B**

Pour $n = 3, \omega = 0.5, \epsilon = 1e - 4$, on obtient que la convergence est atteinte en 34 itérations.

Pour $n = 5, \omega = 1, \epsilon = 1e - 6$ on obtient que la convergence est atteinte en 42 itérations.

Pour $n = 10, \omega = 1.5, \epsilon = 1e - 8$, on obtient que la convergence est atteinte en 54 itérations.

Les tests montrent une comparaison claire entre deux approches pour résoudre des systèmes linéaires. En effet, le programme A, basé sur la décomposition matricielle avec Gauss-Seidel, converge significativement plus rapidement que le programme B, qui implémente directement la formule itérative sans optimisation structurelle pour tous les cas testés. Ainsi, pour le programme A, on constate que le nombre d'itérations reste relativement bas même lorsque n ou ϵ augmentent, et on peut ainsi en déduire que la décomposition $A = D + L + U$ optimise les calculs en isolant les composantes diagonales, triangulaires inférieures et triangulaires supérieures, ce qui augmente l'efficacité numérique et réduit le coût des calculs intermédiaires. À mesure que n augmente, l'écart de performance entre les deux programmes devient plus prononcé, ce qui fait du programme A le choix privilégié pour résoudre des systèmes linéaires de grande taille. Bien que le programme B soit plus simple à implémenter, il est inadapté pour des applications nécessitant des calculs efficaces et précis. Cela met en évidence l'importance d'optimiser les algorithmes numériques pour résoudre efficacement les systèmes linéaires.

2.1.3 Fortran

Pour pouvoir réaliser la méthode de Gauss Seidel en Fortran, il était nécessaire de coder plusieurs sousroutines puisque certaines fonctions qui sont présentes de bases dans Matlab et Python, n'étaient dès lors par présentes en Fortran. Voici donc le TAD du code en Fortran :

- subroutine `matriceCarre(A,carre)` pour vérifier que la matrice A soit bien carre, E : A matrice et S : carre logical
- subroutine `diagonaleDom(A,diag)` pour vérifier que la matrice A soit bien à diagonale dominante, E : A matrice et S : diag logical
- subroutine `GaussSeidelFunction(A,B,Eps,x,m,n)` pour réaliser la méthode de Gauss Seidel, E: A matrice, E: B vecteur, E : Eps real, E/S : x vecteur, S : m integer, E : n integer
- subroutine `MatHilbert(A,w,n)` pour créer la matrice de Hilbert, S : A matrice, E : w real, E : n integer
- subroutine `CalculB(A,B,n)` pour créer le vecteur B, E: A matrice, S : B vecteur, E : n, integer
- subroutine `inverse_matrice(A, A_inv, n)` pour créer l'inverse d'une matrice, E : A matrice, E ; n integer, S : A_inv matrice
- subroutine `diagonale(A,D)` pour créer la diagonale de A, E : A matrice, S : D matrice
- subroutine `triangulaireSupp(A,U)` pour créer la matrice triangulaire supérieure de A, E: A matrice, S : U matrice
- subroutine `triangulaireInf(A,L)` pour créer la matrice triangulaire inférieure de A, E : A matrice, S : L matrice
- subroutine `matrice_multiplication(A, B, C, n)` pour faire la multiplication de deux matrices, E : A,B matrice, E : n integer, S : C matrice
- subroutine `matrice_vect_multiplication(A, b, c, n)` pour faire la multiplication d'une matrice et d'un vecteur, E : A matrice, E : b vecteur, E : n integer, S : c vecteur

Nous allons désormais vérifier si l'hypothèse obtenue lors des tests en Matlab ainsi qu'en Python est vérifiée pour le code en Fortran. Cela nous permettra de nous focaliser seulement sur le programme le plus efficace et de pouvoir faire des tests plus poussés dans

la prochaine partie. On rappelle que le Fortran est un langage de programmation de haut niveau pour des calculs scientifiques et techniques. Il est donc particulièrement adapté pour les calculs scientifiques intenses. Commençons dès lors les tests :

Pour le **programme A**

Pour $n = 3, \omega = 0.5, \epsilon = 1e - 4$, on obtient que la convergence est atteinte en 8 itérations avec un

Pour $n = 5, \omega = 1, \epsilon = 1e - 6$, on obtient que la convergence est atteinte en 9 itérations

pour $n = 10, \omega = 1.5, \epsilon = 1e - 8$, on obtient que la convergence est atteinte en 12 itérations.

Ensuite on a le **programme B**

Pour $n = 3, \omega = 0.5, \epsilon = 1e - 4$, on obtient que la convergence est atteinte en 34 itérations

Pour $n = 5, \omega = 1, \epsilon = 1e - 6$ on obtient que la convergence est atteinte en 41 itérations

Pour $n = 10, \omega = 1.5, \epsilon = 1e - 8$, on obtient que la convergence est atteinte en 53 itérations.

On remarque dès lors que l'hypothèse semble être vérifiée. On a tout comme dernièrement que les deux approches pour résoudre des systèmes linéaires sont tout deux plus ou moins efficaces. Le programme A semble dès lors la meilleure option pour continuer les tests car le programme B est beaucoup moins rapide et efficace. Par ailleurs, on remarque beaucoup plus ici que plus le n augmente, l'écart de performance entre les deux programmes est beaucoup plus prononcé. Finalement, pour continuer les tests, nous allons désormais utiliser le programme A en Fortran.

2.1.4 Comparaison en simple et double précision

Avant de comparer la simple et la double précision, il est nécessaire de rappeler ce que ces dernières sont. La simple et la double précision concernent le niveau de précision des nombres à virgule flottante, c'est-à-dire le nombre de chiffres significatifs ainsi que leur plage d'expression. Ainsi pour la simple précision, cette dernière utilise 4 octets de mémoire donc 32 bits avec 1 bit pour le signe, 8 bits pour l'exposant et 23 bits pour la mantisse. Elle permet de représenter environ 7 chiffres significatifs. Quant à la double précision, cette dernière représente les nombres flottant avec une précision accrue et utilise généralement 8 octets de mémoire, soit 64 bits, avec 1 bit pour le signe, 11 bits pour l'exposant et 52 bits pour la mantisse. Elle permet par ailleurs de représenter 15 chiffres significatifs.

Nous allons ainsi tester dans un premier temps une matrice de Hilbert de taille $n = 8$, avec un $\epsilon = 1e - 8$ et un $\omega = 1.5$ en simple et double précision : Ainsi en simple précision on obtient les résultats suivants :

```

erreur :
  1.68714297
erreur :
  0.784954786
erreur :
  8.98323655E-02
erreur :
  9.50878859E-03
erreur :
  1.34122372E-03
erreur :
  2.07722187E-04
erreur :
  1.56760216E-05
erreur :
  4.17232513E-06
erreur :
  2.38418579E-07
erreur :
  2.38418579E-07
erreur :
  0.00000000
      11      1.00000000      1.00000024      0.999999881      0.999999881      0.999999881      0.999999940

```

FIGURE 1.2 – Simple précision matrice de Hilbert, itérations 11

Et en double précision les résultats suivants :

```

Erreur:
  1.6871428571428575
Erreur:
  0.78495479671963386
Erreur:
  8.9832377427440102E-002
Erreur:
  9.5087632745053385E-003
Erreur:
  1.3411286731521699E-003
Erreur:
  2.0773238524440796E-004
Erreur:
  1.5677168016647869E-005
Erreur:
  4.1661334874154576E-006
Erreur:
  2.0860703231484479E-007
Erreur:
  7.5166560975503671E-008
Erreur:
  4.1023520136462821E-009
      11      0.9999999871491163      0.9999999983622478      1.0000000001500460      1.0000000002142970
00000691081

```

FIGURE 1.3 – Double précision matrice de Hilbert, itérations 11

On remarque dès lors que la double précision est bien plus précise puisqu'à la dernière itération, par exemple, on a une erreur qui est de 0.000000 pour la simple précision, alors que pour la double, on a une erreur de l'ordre de $1e-9$ avec $4.1023520136462821E-9$. Par ailleurs, le nombre de chiffres significatifs de la solution x est beaucoup plus important en double précision, alors qu'on remarque que pour la simple, certaines des valeurs sont arrondies. On peut par ailleurs observer que les erreurs diminuent mais stagnent à une valeur autour de $1e-3$ à $1e-4$. Cela reflète bien la limite de précision, due à l'utilisation de nombres en simple précision, puisque lorsqu'on utilise la double précision, les erreurs

diminuent beaucoup plus rapidement, atteignant des valeurs autour de $1e-7$. Ainsi, on peut en conclure que, si la précision est cruciale dans les applications que l'on souhaite utiliser, il est préférable d'utiliser la double précision, alors que, si l'on souhaite simplement utiliser des applications moins sensibles, il peut être intéressant d'utiliser la simple précision.

Nous allons désormais comparer la simple et la double précision sur deux autres matrices, bien que ces dernières ne soient pas à diagonale dominante. Hypothétiquement, on ne peut pas assurer que la méthode de Gauss-Seidel converge. Cependant, il est tout de même possible qu'il y ait convergence.

Soit la matrice A et le vecteur b définis comme tels :

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 5 & 1 & 2 & 3 & 4 \\ 4 & 5 & 1 & 2 & 3 \\ 3 & 4 & 5 & 1 & 2 \\ 2 & 3 & 4 & 5 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \\ 3 \\ 4 \\ 2 \end{pmatrix},$$

On a ainsi en simple précision les résultats suivants avec toujours un $\epsilon = 1e-8$:

```

erreur :
  78.0000000
erreur :
 19808.0000
erreur :
 5076358.00
erreur :
 1.30095590E+09
erreur :
 3.33405585E+11
erreur :
 8.54443227E+13
erreur :
 2.18974505E+16
erreur :
 5.61182214E+18
erreur :
 1.43818330E+21
erreur :
 3.68573909E+23
erreur :
 9.44571710E+25
erreur :
 2.42072378E+28
erreur :
 6.20376847E+30
erreur :
 1.58988562E+33
erreur :
 4.07451742E+35
erreur :
 1.04420680E+38
erreur :
  Infinity
erreur :
      NaN      NaN      NaN      NaN      NaN
18

```

FIGURE 1.4 – Simple précision matrice A et vecteur b

Et en double précision on a les résultats suivants :



FIGURE 1.5 – Double précision matrice A et vecteur b .

On remarque dès lors qu'il n'y a pas convergence et par ailleurs, l'erreur ne fait que d'augmenter que ce soit en simple ou bien en double précision. Il n'est donc pas étonnant de voir qu'il n'y a pas convergence puisque l'on est passé outre le fait que la matrice A soit à diagonale dominante.

De même avec les matrices :

$$A' = \begin{pmatrix} 1 & -1 & -200 & 0 \\ 0 & 1 & 100 & -100 \\ 0 & 1 & 101 & -101 \\ 0 & 0 & 0 & 100 \end{pmatrix}, \quad b' = \begin{pmatrix} 1 \\ 1 \\ 2 \\ 4 \end{pmatrix}.$$

On a ainsi en simple précision :

```

7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
1.52587891E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
1.52587891E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
1.52587891E-05
erreur :
7.62939453E-06
erreur :
2.28881836E-05
erreur :
1.52587891E-05
erreur :
0.00000000
1204    109.999748    -98.9997482    1.03999746    3.99999991E-02

```

FIGURE 1.6 – Simple précision matrice A' et vecteur b' , itération 1204.

Et en double précision :

```

1.2855437603320752E-008
Erreur:
1.2728122555927257E-008
Erreur:
1.2602129118022276E-008
Erreur:
1.2477343602768087E-008
Erreur:
1.2353808642728836E-008
Erreur:
1.2231481605340377E-008
Erreur:
1.2110376701457426E-008
Erreur:
1.1990479720225267E-008
Erreur:
1.1871776450789184E-008
Erreur:
1.1754210049730318E-008
Erreur:
1.1637851571322244E-008
Erreur:
1.1522629961291386E-008
Erreur:
1.1408531008783029E-008
Erreur:
1.1295597346361319E-008
Erreur:
1.1183743708897964E-008
Erreur:
1.1072998518102395E-008
Erreur:
1.0963390195684042E-008
Erreur:
1.0854833476514614E-008
Erreur:
1.0747356782303541E-008
Erreur:
1.0640945902196108E-008
Erreur:
1.0535600836192316E-008
Erreur:
1.0431278951728018E-008
Erreur:
1.0328008670512645E-008
Erreur:
1.0225718938272621E-008
Erreur:
1.0124509230990952E-008
Erreur:
1.0024251650975202E-008
Erreur:
9.9249888307895162E-009
1857 109.99999901742480 -98.999999007499810 1.0399999901732655 4.000000000000000

```

FIGURE 1.7 – Double précision matrice A' et vecteur b' , itérations 1857.

On remarque qu'il y a convergence, cependant cette dernière est atteinte en un très grand nombre d'itérations. Par ailleurs, on remarque qu'en double précision, le nombre d'itérations pour atteindre la convergence est beaucoup plus important, ce qui peut sembler contre intuitif aux premiers abords mais qui peut s'expliquer par le fait qu'on a un seuil d'arrêt plus strict et une propagation des erreurs réduites. Les erreurs d'arrondi et de troncature en double précision sont significativement plus petites qu'en simple précision. Ainsi la convergence observée, bien que lente, montre la capacité de la méthode à résoudre

le système malgré une matrice mal conditionnée. Par ailleurs, le nombre élevé d'itérations en double précision illustre la recherche d'une précision accrue mais elle met tout autant en évidence le coût numérique associé. Cela souligne l'importance des conditions, comme la condition de diagonale dominante, pour résoudre des systèmes linéaires complexes.

Conclusion

Au cours de ce projet, nous avons pu explorer en détail la méthode de Gauss-Seidel et ses performances dans la résolution de systèmes linéaires pour différentes matrices. Nous avons ainsi pu constater en quoi le conditionnement d'une matrice est important selon la méthode utilisée avec l'exemple des deux matrices à diagonales non dominantes. Par ailleurs, nous avons pu avoir une analyse comparative entre la simple et la double précision, ce qui nous a permis de comprendre l'impact de la précision numérique. La méthode de Gauss-Seidel reste donc une technique fondamentale et instructive dans le cadre de la résolution de systèmes linéaires. Cependant, son efficacité dépend fortement des propriétés des matrices et des contraintes numériques associées. Le fait d'avoir utilisé la méthode de Gauss-Seidel a permis de mettre en exergue qu'il est important d'adopter une approche adaptée à chaque problème, en combinant une analyse théorique ainsi que des choix méthodologiques pour garantir une résolution optimale. Pour conclure, ce projet nous a permis de découvrir des perspectives intéressantes pour l'exploration de méthodes complémentaires et la mise en œuvre d'optimisations dans des contextes plus complexes ou exigeants.